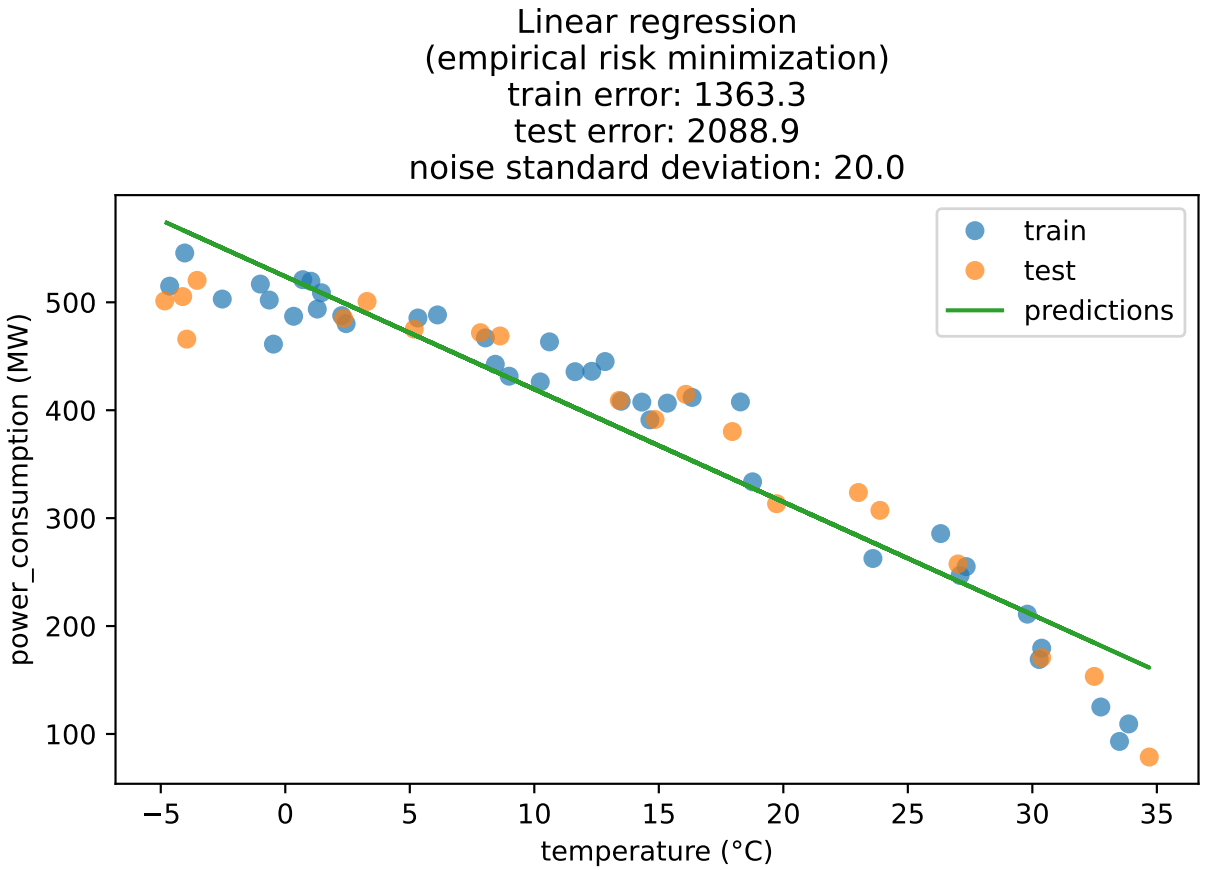
FTML: train and test set, regression.



Content

Train sets and test sets, loss functions

Linear regression in one dimension

Linear regression

Linear regression is one of the most elementary methods used in ML regression problems. It is useful for many applications, and is often a component of more complex methods.
We will use is to illustrate several important aspects of ML that are also encountered when using other methods (kernels, trees, neural networks, etc.), namely :

- the train and test sets.
- overfitting (when the dimension d is not really small).

Example problem

We want to predict the power that needs to be produced by a power plant in a city, as a function of the temperature only.

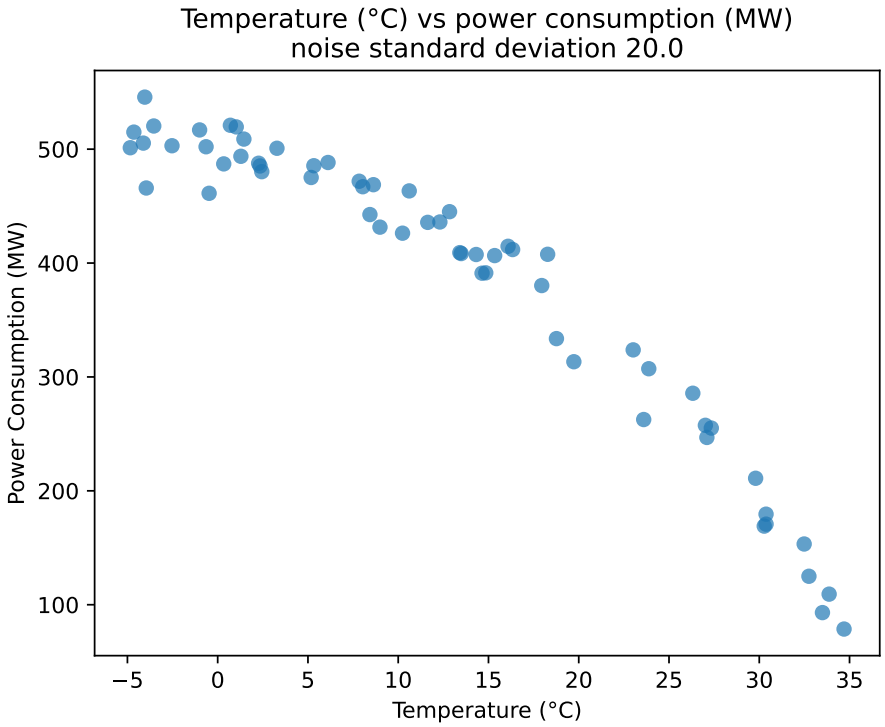


Figure – Dataset

The only information we have is a collection of n **samples**, called the **dataset**. Each sample consists in two values :

- the temperature in °C (the **feature**).
- the power consumption in W (the **label**).

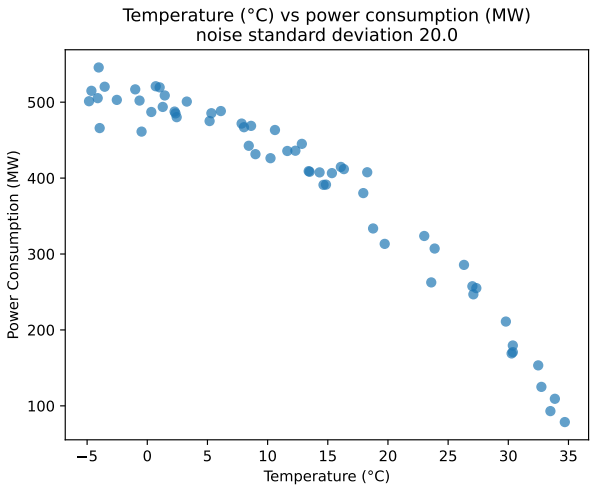


Figure – Dataset

Exercice 1 : What is d in this example ?

Supervised learning

Based on this **finite** dataset, our objective is to produce a function, noted $\tilde{f}$, called the **model**, the **predictor**, or the **estimator**, that will map a temperature to a power consumption.

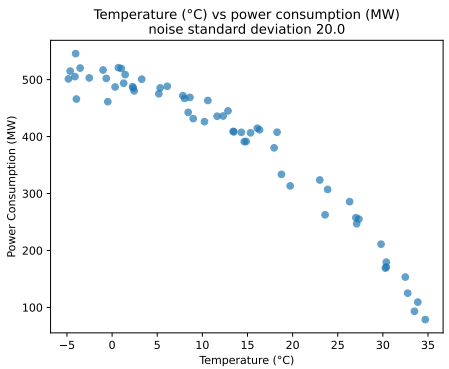


Figure – Dataset

The problem of supervised learning is that we want this function to perform well on **new, previously unseen samples**. It is not useful in itsself to perform well on the dataset !

Supervised learning

Based on this **finite** dataset, our objective is to produce a function, noted $\tilde{f}$, that will map a temperature to a power consumption.

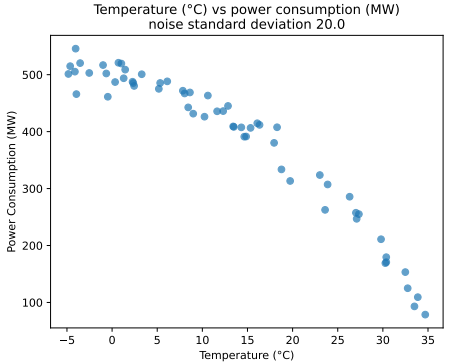


Figure – Dataset

Exercice 2 : How could we **estimate** the performance of $\tilde{f}$ on unseen data, using the dataset ?

Train set and set set

We split the dataset in two parts :

- the **train set** will be used to learn (train $\tilde{f}$)
- the **test set** will be used to estimate the performance of $\tilde{f}$ on unseen data.

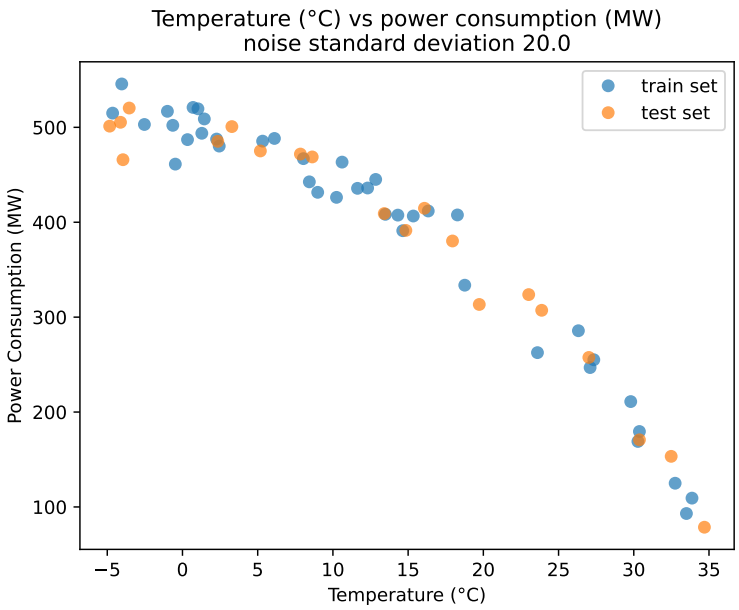


Figure – Splitted Dataset

Loss functions

To evaluate the performance of $\tilde{f}$, we use a **loss function**. The loss function is be a measure of the **discrepancy** between a prediction $\tilde{f}(x)$ and a label y (that are both real numbers). In regression, the most common loss function is the **squared loss** :

$$(\tilde{f}(x) - y)^2 \tag{1}$$

Averaging over the whole dataset, this leads to :

- The train error

$$\frac{1}{n_{\text{train}}} \sum_{(x_i, y_i) \in (X_{train} \times y_{train})} (\tilde{f}(x_i) - y_i)^2 \tag{2}$$

- The test error

$$\frac{1}{n_{\text{test}}} \sum_{(x_i, y_i) \in (X_{test} \times y_{test})} (\tilde{f}(x_i) - y_i)^2 \tag{3}$$

Empirical risk minimization

The most common supervised learning process is then naturally to :

- Find $\tilde{f}$ the as a small training error.
- Compute the test error of $\tilde{f}$ in order to estimate its performance on unseen data.

Remark : many of these two aspects will be discussed more deeply later in the couse (e.g. notions of validation set, or optimization error)

Train sets and test sets, loss functions

Linear regression in one dimension

Linear regression

The remaining step is then to find $\tilde{f}$. The first step, called **model selection**, is to decide the type of function / model use. Many types of models exist, each one having potential drawbacks, for instance :

- linear functions (which we will use in this first example).
- polynomial functions
- kernels
- neural networks
- support vector machines

`https://scikit-learn.org/stable/machine_learning_map.html`

Exercice 3 : Why are the samples not on a line (and not on a curved line either) ?

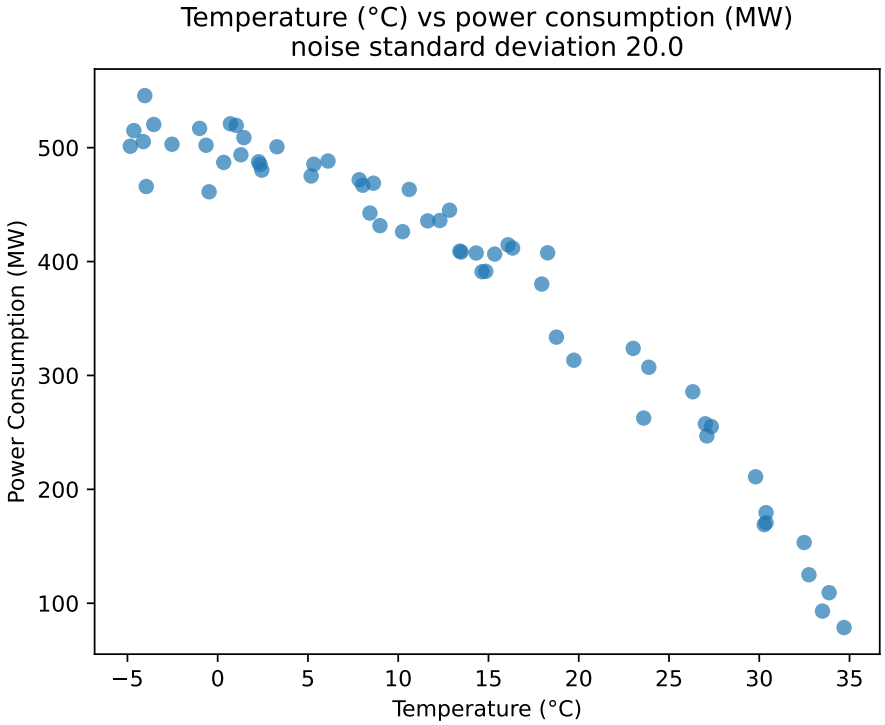


Figure – Dataset

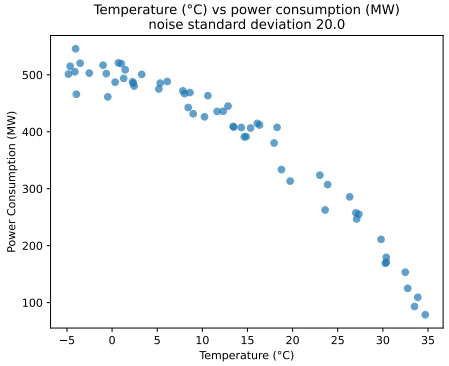


Figure – Dataset

The power consumption does not depend **only** on the temperature, but also on many other variables, that we do not have access to here :

- time in the day
- humidity, wind
- period of the year (holidays or not)
- other variables

Linear regression

Formalization :

- input space (temperature in °C) : $\mathcal{X} = \mathbb{R}$
- output space (power consumption in W) : $\mathcal{Y} = \mathbb{R}$
- dataset : $D = \{(x_1, y_1), \ldots, (x_n, y_n), i \in [1, n]\}$.

linear regression in dimension 1 (as $x \in \mathbb{R}$), means that our estimator is of the form :

$$\tilde{f}(x) = \theta x + b \tag{4}$$

with $\theta \in \mathbb{R}$, $b \in \mathbb{R}$.

Empirical risk minimization

With the squared loss, and this function $\tilde{f}$ the **train error** is :

$$R_{n_{train}}(\theta, b) = \frac{1}{n_{train}} \sum_{(x_i, y_i) \in (X_{train} \times y_{train})} (\theta x_i + b - y_i)^2 \qquad (5)$$

The **optimization problem** is to find θ and b such that the empirical risk $R_{n_{train}}(\theta, b)$ has the **smallest possible value**.

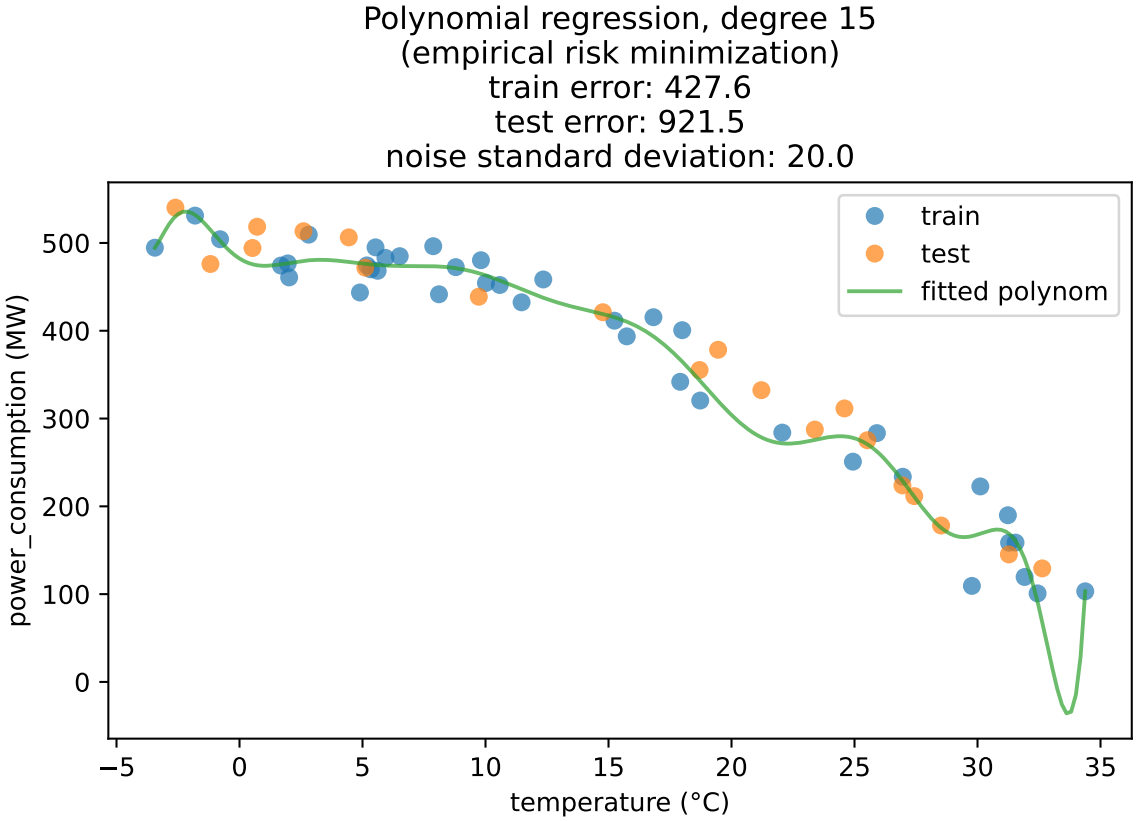
Analytic solutions

For some problems, like this one, it is possible to explicitely compute the optimal solution.
Thanks to the convexity and differentiability of $R_n(\theta)$ with respect to θ, the points optimizing the empirical risk are obtained by finding (θ^*, b^*) such that the gradient cancels (more on that later in the course).

$$\nabla_{(\theta,b)} R_n(\theta^*, b^*) = 0 \tag{6}$$

Computing the optimal values

Exercice 4 : Compute the gradient and find the values θ^* and b^* that cancel it.

Overfitting

In the first practical session, we will observe the interplay between n, d, and the amount of overfitting.